

Inteligencia artificial avanzada para la ciencia de datos

Alejandro Adrián García Martínez – A01707109

Escuela de ingeniería y Ciencias

30 de septiembre del 2026

Abstracto— Este documento se enfoca en la implementación manual de un algoritmo de regresión logística utilizando una gradiente descendiente sin la asistencia de bibliotecas externas. El objetivo es desarrollar un modelo capaz de predecir con precisión si un alumno es un caso de depresión en base a distintos parámetros en su vida diaria. Se realizó un Análisis Exploratorio de Datos (EDA) para entender y limpiar los datos y permitir que el modelo los pueda interpretar de forma correcta

I. INTRODUCCIÓN

La inteligencia artificial ha evolucionado a pasos agigantados en los últimos años, pasando a volverse una herramienta de uso diario por millones de personas alrededor del mundo. Originalmente se utilizaba para crear sistemas que pudieran realizar tareas que requerían de la intervención humana, hoy en día esta se ha diversificado en distintos campos de investigación en diferentes áreas del conocimiento, uno de ellos el conocido como “*Machine Learning*”.

Este tipo de inteligencia artificial es capaz de aprender de los datos y mejorar su rendimiento sin tener una tarea en concreto dándole flexibilidad para poder ser utilizada en un sinnúmero de proyectos. Esta busca patrones para poder tomar decisiones y se clasifica en 3 categorías: aprendizaje supervisado, no supervisado y por esfuerzo. El modelo presentado en este documento es de tipo “*Aprendizaje Supervisado*” en el que se entrena un modelo con un conjunto de datos y se sabe la respuesta correcta, el modelo intenta predecir la respuesta y se compara la predicción con la real, finalmente se ajustan los parámetros del modelo para acercarse a acertar con mayor frecuencia.

II. CONTEXTO DEL PROBLEMA

A. Origen

Los datos por utilizar en este documento son recopilados de la página “*Kaggle*”. Estos datos recopilan las respuestas de distintos jóvenes de diferentes ciudades y contestan preguntas relacionadas a sus hábitos y esfuerzo en los estudios o trabajo.

Algunos de estos jóvenes sufren de depresión un trastorno mental que afecta la salud emocional y mental del individuo que lo sufre, algunos de los síntomas conocidos son sentimientos de tristeza, fatiga, cambios en el sueño, alteraciones en el apetito, entre otros. De no ser tratado de forma correcta puede llevar al individuo a la muerte.

El objetivo de estos datos es poder entrenar un modelo de “*Machine Learning*” que al conocer la respuesta a ciertos hábitos de un individuo pueda predecir si la persona sufre de depresión.

B. Base de datos

Los datos consisten en 27,901 instancias repartidas en 18 columnas o “*features*”. Estas columnas representan la información de la siguiente forma:

Id	Identificador único	Identificador único
Gender	Genero del individuo	Male Female
Age	Edad del individuo	Int64
City	Ciudad donde reside	Str
Profession	A que se dedican	Str
Academic Pressure	Cuanta presión sienten por parte de la escuela	0 a 5 0: Poco o nada 5: Mucho
Work Pressure	Presión por parte del trabajo	0 a 5 0: Poco o nada 5: Mucho
CGPA	Promedio académico	Float64
Study Satisfaction	Satisfacción con sus estudios	0 a 5 0: Poco o nada 5: Mucho
Job Satisfaction	Satisfacción con su trabajo	0 a 5 0: Poco o nada 5: Mucho
Sleep Duration	Duración del sueño	Less than 5 hours 5-6 hours 7-8 hours More than 8 hours
Dietary Habits	Hábitos alimenticios	Unhealthy Moderate Healthy

Degree	Grado académico	Str
Have you ever had suicidal thoughts?	Si el individuo ha tenido pensamientos suicidas de forma reciente	Yes No
Work/Study Hours	Horas dedicadas a los estudios o al trabajo	Float64
Financial Stress	Estrés financiero	0 a 5 0: Poco o nada 5: Mucho
Family History of Mental Illness	Anteriormente familiares han sufrido de problemas mentales	0: No 1: Yes
Depression	El objetivo que predecir	0: No 1: Yes

C. Análisis Exploratorio de Datos

El objetivo del “EDA” es conocer, entender y limpiar la estructura del conjunto de datos utilizado en este proyecto. En este proceso se identifican los valores nulos, las columnas que generan un mayor impacto y las columnas que no aportan al aprendizaje del modelo.

Este análisis se realizó en el archivo “eda.py” que se encuentra en este mismo repositorio, ahí se encuentra el proceso, así como las gráficas generadas como la toma de decisión en tiempo real sobre los hallazgos encontrados. A continuación, se dará un análisis de lo más relevante.

1) Valores nulos, únicos y duplicados

Lo primero que se buscó dentro del data set fue la existencia de valores duplicados, estos se caracterizan por ser idénticos en todas las columnas a otras instancias dentro del set de datos. Esto puede afectar gravemente el aprendizaje del modelo ya que no está aprendiendo de distintos datos y no le permite ajustarse de forma correcta, provocando que haga sus predicciones de forma incorrecta. En el caso de este set de datos no se encontraron datos duplicados indicando que las 27,901 instancias son únicas.

Después de revisar el tipo de dato de cada columna y como se llamaban se buscó que datos se encontraban como datos nulos o que aparecieran con el símbolo de interrogación “?”. Solamente se encontraron 3 de estos datos dentro de nuestro set, dejándonos con 27,899 instancias únicas y completas.

Finalmente nos adentramos a identificar que tipo de dato existe en cada columna, cuales son los más frecuentes y cuales son los más alejados. Primeramente, la columna de “id” como se mencionó en el apartado anterior cuenta con un identificador único de cada instancia, este identificador no aporta en el resultado final de la predicción y puede generar ruido para el modelo, es por eso que se decide eliminar la columna.

```
===== Conteo de Datos por columna =====
id
2      1
8      1
26     1
30     1
32     1
140685 1
140686 1
140689 1
140690 1
140699 1
Name: count, Length: 27901, dtype: int64
```

Fig. 1 Conteo de valores en la columna “id”

2) Valores poco elegidos

Lo segundo fue revisar el conteo de las demás columnas, con esto se pudo identificar que en la columna de “Profession” alrededor de 27,870 de los datos eran alumnos, la gran mayoría, por lo que esta columna no aporta un gran peso al resultado final ya que casi todos los participantes eran únicamente estudiantes y las demás profesiones no aportan un valor significativo a los datos, por lo que se decide eliminar la columna.

```
/=/=/=/=/=/=/=/=/=/=
Profession
Student      27870
Architect    8
Teacher      6
Digital Marketer 3
Content Writer 2
Chef         2
Doctor       2
Pharmacist   2
Civil Engineer 1
UX/UI Designer 1
Educational Consultant 1
Manager      1
Lawyer       1
Entrepreneur 1
Name: count, dtype: int64
/=/=/=/=/=/=/=/=/=/=
```

Fig. 2 Conteo de valores en la columna “Profession”

Como la mayoría de los datos indican que son estudiantes un caso similar sucede en las columnas relacionadas al trabajo que son “Work Pressure” y “Job Satisfaction”. Donde casi todas las respuestas indicaron 0. Es por esto que ambas columnas se deciden eliminar por el mismo motivo.

```
/=/=/=/=/=/=/=/=/=/=
Work Pressure
0.0      27898
5.0       2
2.0       1
Name: count, dtype: int64
/=/=/=/=/=/=/=/=/=/=
```

Fig. 3 Conteo de valores en la columna “Work pressure”

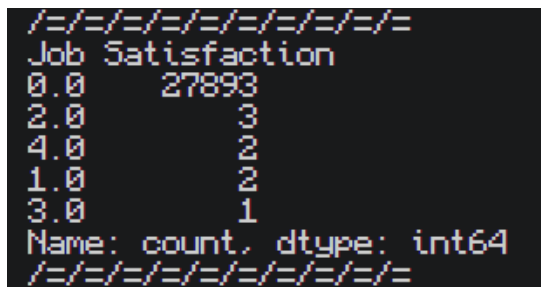


Fig. 4 Conteo de valores en la columna "Job Satisfaction"

La columna de "Work/Study Hours" generaliza ambos tipos de respuesta por lo que se puede conservar, si estuvieran separados se borraría la relacionada al trabajo.

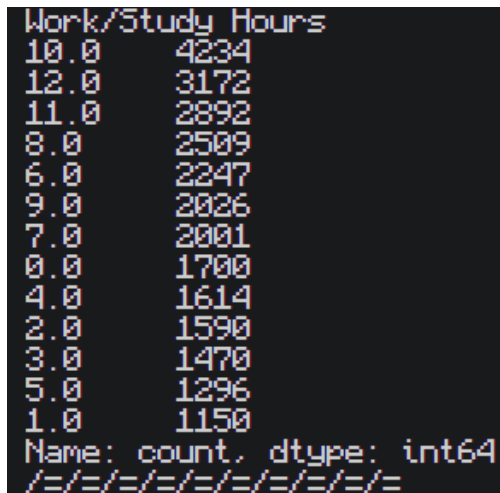


Fig. 5 Conteo de valores en la columna "Work/Study Hours"

3) Datos tipo "str"

Para las columnas de "City" y "Degree" se hizo profundizo un poco más sobre que representaban estos datos y si convenía mantenerlos y de que forma. Se realizó la siguiente grafica para poder tener una representación visual de cómo se comportan los datos.

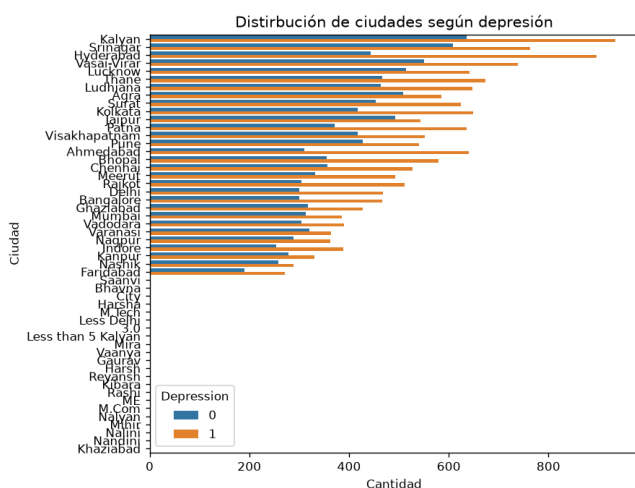


Fig. 6 Distribución de ciudades según depresión

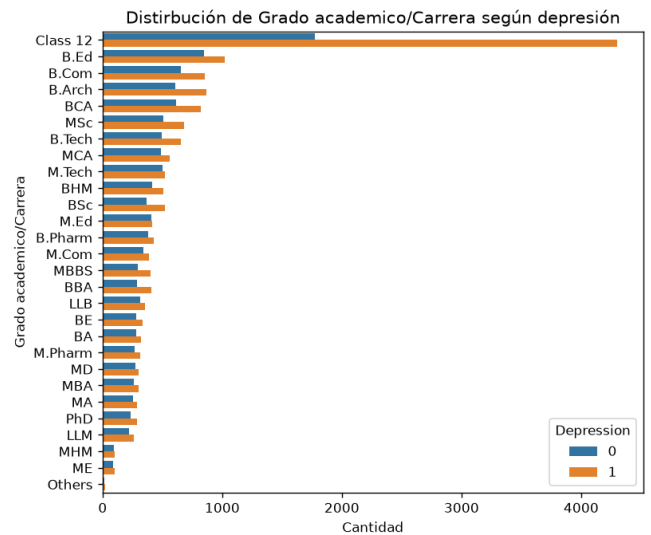


Fig. 7 Distribución de carreras según depresión

Como se puede ver en la gráfica de las carreras la gran cantidad de los datos se encuentran en el valor "Class 12" esto nos indica no realmente que si se encuentra en la clase 12 tiende a tener más probabilidades de presentar depresión, si no, como fueron recabados los datos. Recordemos que la gran mayoría son estudiantes por lo que pocos han de tener un rango académico mayor a estudiante, como se puede ver en la gráfica. Por lo que estos datos nos indican como fue tomada la muestra no un indicador real.

Algo similar sucede con las ciudades, esos datos representan las regiones geográficas donde se recopilaron los datos no indican que al provenir o residir en cierta ciudad es más probable de padecer depresión. Es por eso que se tomó la decisión de eliminar ambas columnas, también para lograr que el modelo pudiera hacer el análisis enfocándose en la forma de vida más que en el lugar que se reside.

4) Datos extremos

Finalmente se identificó que la distribución de datos en la columna de tenía algunos valores extremos.

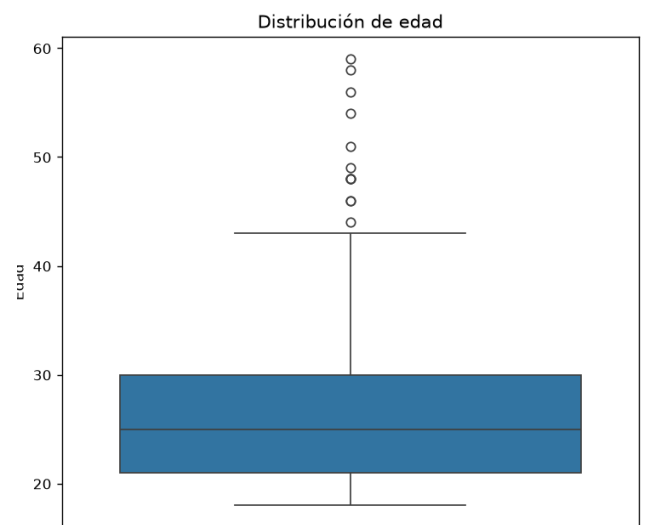


Fig. 8 Distribución de edad

Como se puede ver en la caja de bigotes la mayoría de edad esta entre los 21 años y los 30, pero hay registros de hasta 56 años. Para mantener los datos centrados y que se apegue al perfil de un estudiante se decidió mantener los datos en un rango de máximo 30 años.

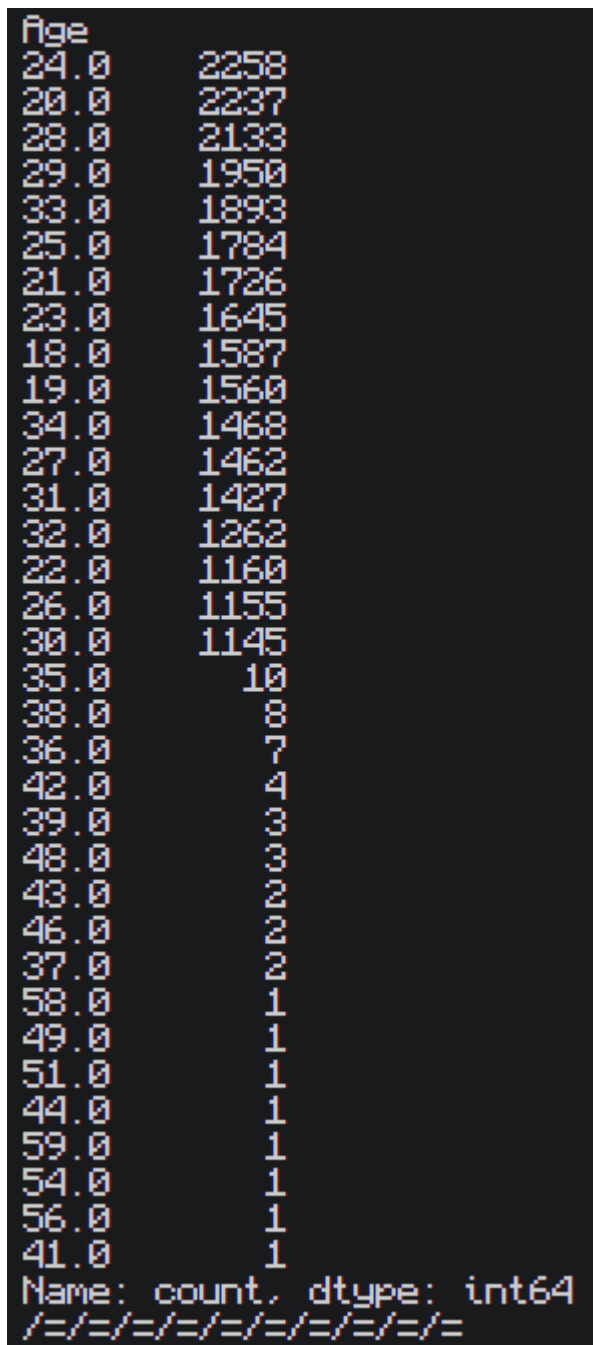


Fig. 9 Conteo de la columna "Age"

5) Transformación de los datos

Después de hacer el análisis y decidir que columnas se quedarán, se transformaron los datos para poder mantener todo en valores numéricos. Es decir, columnas como "Sleep duration" pasarían de tener valores como "Menos de 5 horas" a un valor numérico como "0". Esto para que al modelo le sea más fácil leer los datos de clasificación.

```

===== Info del Dataset =====
<class 'pandas.DataFrame'>
Index: 21802 entries, 1 to 27900
Data columns (total 12 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Gender                                21802 non-null  int64
1   Age                                   21802 non-null  float64
2   Academic Pressure                     21802 non-null  float64
3   CGPA                                  21802 non-null  float64
4   Study Satisfaction                    21802 non-null  float64
5   Sleep Duration                        21790 non-null  float64
6   Dietary Habits                        21793 non-null  float64
7   Suicidal Thoughts                     21802 non-null  int64
8   Work/Study Hours                      21802 non-null  float64
9   Financial Stress                      21800 non-null  float64
10  Family Illness                        21802 non-null  int64
11  Depression                            21802 non-null  int64
dtypes: float64(8), int64(4)

```

Fig. 10 Columnas del set de datos transformados

Al hacer la transformación hay datos que se clasificaban como otros, al ser muy pocos en todos los datos se decidieron eliminarlos.

```

===== Datos faltantes =====
Null data:
Gender                                0
Age                                   0
Academic Pressure                     0
CGPA                                  0
Study Satisfaction                    0
Sleep Duration                        12
Dietary Habits                        9
Suicidal Thoughts                     0
Work/Study Hours                      0
Financial Stress                      2
Family Illness                        0
Depression                            0
dtype: int64
Null data: 23
/=/=/=/=/=/=/=/=/=/=/=

```

Fig. 11 Datos otros transformados en "null data"

Finalmente, el set de datos tras ser analizado y limpiado queda con 21,779 en 11 columnas de atributos y 1 columna que funciona como el objetivo a predecir.

III. MODELO DE PREDICCIÓN

A. Presentación del modelo

Tras el extenso análisis de los datos y su limpieza es momento de elegir un modelo de predicción para resolver este enigma. Por la naturaleza de los datos a primera instancia se hace notar que el valor que intentamos predecir es 1 o 0, es decir, categórico por lo que el modelo que mejor se ajusta a estas situaciones es la regresión logística.

Este modelo consiste en aprovechar el modelo lineal y aplicar la función de la sigmoide, la cual hace que el resultado entre en un rango de 1 y 0, justo lo que necesitamos para este problema. La ecuación de la sigmoide es la siguiente.

$$f(x) = \frac{1}{1 + e^{-x}}$$

Fig. 12 Ecuación de la sigmoide

De esta manera se pueden lograr 2 cosas, primero como ya se mencionó el resultado entra en un rango de 1 a 0, el segundo es que la gráfica toma una forma distinta que se ajusta a los datos de una mejor forma.

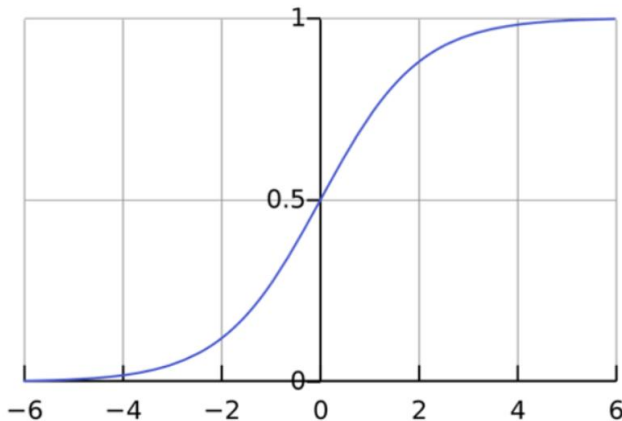


Fig. 13 Función de regresión logística

A diferencia de la regresión lineal en la que en ocasiones algunos datos eran registrados de forma incorrecta por la manera que los captaba arriba o debajo de la línea, la regresión logística permite establecer un umbral para determinar a qué categoría pertenece cada dato de manera más eficiente. De esta forma, el modelo puede separar las observaciones en las clases correspondientes y realizar predicciones más adecuadas para la naturaleza binaria de nuestro problema.

B. Optimización

Uno de los elementos más importantes del entrenamiento es el método de optimización. Al inicio del entrenamiento, nuestro algoritmo no tendrá los valores correctos para los pesos de cada variable, por lo que necesitamos un método que permita ajustarlos de manera adecuada para obtener mejores predicciones.

La gradiente descendiente es un método de optimización utilizado para minimizar la función de pérdida, ajustando los parámetros del modelo, en este caso, los pesos y el sesgo. En cada iteración, el algoritmo calcula la gradiente de la función de pérdida, la cual indica la dirección en la que se debe modificar cada parámetro para reducir el error. De esta manera, los pesos se actualizan progresivamente con el objetivo de disminuir la función de pérdida y acercarla lo más posible a cero.

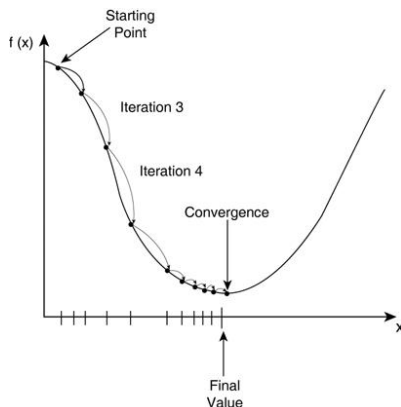


Fig. 14 Gradiente descendiente

C. Cross-Entropy

Cross-Entropy o entropía cruzada es una función que permite medir la diferencia entre dos distribuciones de probabilidad. Es decir, permite comprar que tan alejado esta la predicción de la realidad. En el caso de regresión logística, el modelo genera una probabilidad entre 0 y 1 para cada observación. La función penaliza predicciones incorrectas con un valor alto como resultado por lo que nuevamente buscamos llegar lo más cercano a 0 posible. La utilizaremos como función de pérdida para evaluar qué tan bien está prediciendo el modelo con respecto a la realidad a lo largo de las iteraciones.

$$\mathcal{L}(\theta) = -\frac{1}{n} \sum_{i=1}^n y_i \log(p_i) + (1 - y_i) \log(1 - p_i)$$

Fig. 15 Función de cross-entropy

IV. PRIMER MODELO

A. Preparación de los datos

Para poder entrenar de forma correcta y sacar el mayor provecho a los datos que tenemos disponibles es imprescindible separarlos en distintos conjuntos que permitirán asegurar la mayor precisión de predicción por parte del modelo.

Estos conjuntos serán conocidos como “train”, “validation” y “test”. Hablemos un poco del propósito de cada uno de ellos.

- **Train:** Este conjunto de datos será el más grande y permitirá el modelo a ser entrenado y ajustar su precisión con las ecuaciones anteriormente mencionadas. Está conformado por el 80% de los datos
- **Validation:** Este conjunto nos sirve para evaluar el modelo durante el entrenamiento, nos ayudará a decidir que tan bien se ajusta el modelo al problema y detectar problemas de “overfit” o “underfit”. Es importante mencionar que estos datos no los utiliza durante su entrenamiento. Está conformado por el 10% de los datos
- **Test:** Este conjunto de datos nos permite calcular la precisión del modelo después de haber sido entrenado y con datos que no ha visto antes. Esto permite ver que tan bien funciona el modelo y que tan bien se ajustó. Está conformado por el 20% de los datos

Así mismo en todos los conjuntos se separan las columnas de la columna denominada “target” originalmente, estas serán la “X” y “Y”.

B. Descripción del modelo

Nombre	Función	Descripción
sigmoid(z)	Función de activación	Convierte el valor calculado

		en una probabilidad entre 0 y 1.
Cross-entropy(X, y, w, b)	Función de costo	Calcula el error del modelo mediante la entropía cruzada.
grad_fun(X, y, w, b)	Cálculo del gradiente	Calcula los gradientes necesarios para actualizar los pesos y el sesgo.
grad_des(x_train, y_train, x_val, y_val, alpha, iterations)	Función de optimización	Ajusta los pesos mediante descenso del gradiente y registra el error de entrenamiento y validación.
predict(X, w, b)	Predicción	Genera las predicciones finales del modelo, clasificando cada dato como 0 o 1.

C. Parámetros

Pesos (w): Son inicializados en 0 y actualizados durante el entrenamiento dentro de la función de optimización.

Sesgo (b): Inicializado en 0 y actualizado durante el entrenamiento dentro de la función de optimización.

D. Hiperparámetros

Learning rate (lr): Taza de aprendizaje, controla que tan grandes son los pasos de la gradiente descendiente. Esta inicializado en 0.1.

Epochs (iteraciones): Número de iteraciones para ajustar los pesos del modelo. Una sola época equivale a una vuelta completa de los datos en la función de optimización. Se harán dos pruebas una con 1000 iteraciones y otra con 500 iteraciones.

V. RESULTADOS DEL PRIMER MODELO

A. Resultados

El primer modelo fue entrenado en dos configuraciones diferentes, utilizando 1000 iteraciones y 500 iteraciones. Los resultados muestran que a partir de las 200 iteraciones el modelo se ha acercado bastante a su punto de estabilización.

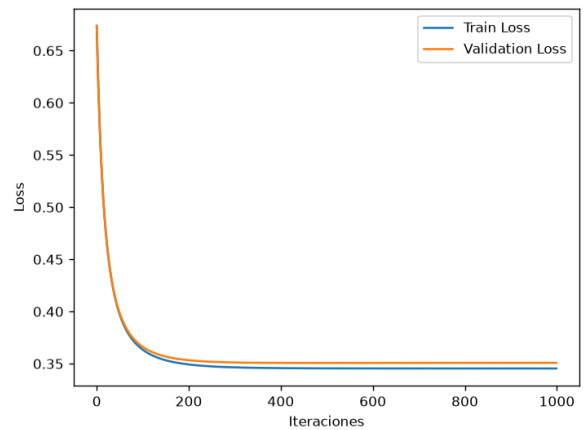


Fig. 16 Grafica de costos train contra validation en 1000 iteraciones

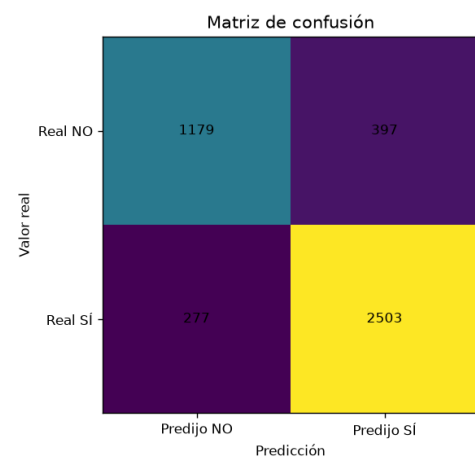


Fig. 17 Matriz de confusión del primer modelo en 1000 iteraciones

La precisión del primer modelo en 1000 iteraciones usando los datos de "test": 84.52708907254362.

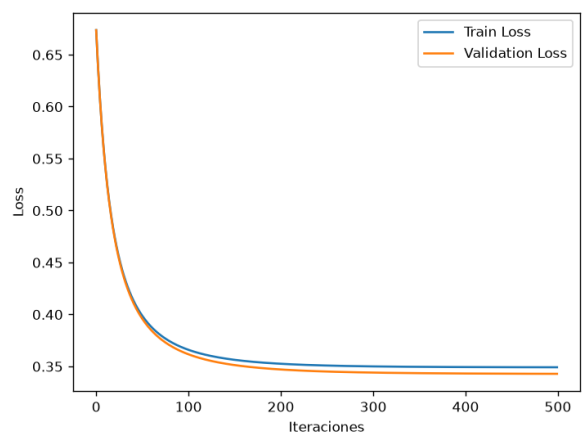


Fig. 18 Grafica de costos train contra validation en 500 iteraciones

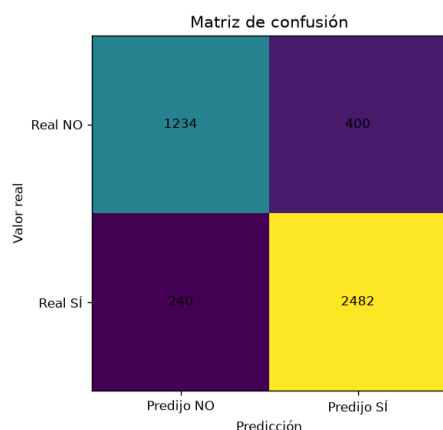


Fig. 19 Matriz de confusión del primer modelo en 500 iteraciones

La precisión del primer modelo en 500 iteraciones usando los datos de “test”: 85.30762167125803.

B. Interpretación

Como ya se mencionó el modelo alcanza una rápida disminución de la función de costo durante las primeras iteraciones para estabilizarse alrededor de las 200. Las curvas de “train” y “validation” permanecen cercanas por lo que no se observa evidencia de “overfit”. En cuanto al desempeño final en ambas versiones se alcanza más del 80% de precisión en las predicciones de “test”, la diferencia entre ambas versiones es de solamente un 0.8% aproximadamente.

En cuanto a los errores de predicción podemos ver que ambas versiones tienen una tendencia de predecir más falsos positivos (FP), en nuestro contexto significa que habría casos que personas recibirían tratamiento de depresión cuando no la tiene realmente, en cambio de los falsos negativos (FN) que son los que más nos interesa reducir pues estas personas están en riesgo de salud mental y no están recibiendo tratamiento.

C. Diagnóstico

La precisión entre los dos modelos como ya se mencionó es mayor al 80%, así mismo los errores recen más en falsos positivos esto sugiere que el modelo si se está adaptando a los datos. En nuestras graficas de costo, podemos ver que dejan de bajar considerablemente y tanto entrenamiento como validación se mantiene juntas por lo que podemos descartar una situación de “overfitting”. Sin embargo, ambos se mantienen por arriba del 0.34 lo cual puede indicar una situación de “underfitting” pues hay cosas que el modelo no está ajustando de forma correcta o hay una falta de parámetros que permitan al modelo tener un costo menor, es importante probar con otro modelo para verificar si este valor de costo es lo más bajo o nos encontramos en una situación de “underfitting”.

VI. MODELO CON FRAMEWORK (RANDOM FOREST TREE)

Para verificar y comparar el desempeño del primer modelo de regresión, es importante compararlo con el desempeño de otro modelo. Haremos uso de un “framework” en esta ocasión, con la idea de utilizar un modelo más optimizado y estandarizado que una implementación desde 0, así sabremos si se puede predecir correctamente el problema y como ajustar el primer modelo.

A. Contexto

“Random Forest Trees” es un conjunto de métodos de ML que aprovecha la idea de los árboles de decisión y la expande creando múltiples árboles de decisión para mejorar la precisión y robustez, por eso su nombre de bosque. La idea central del bosque es que múltiples árboles entrenados de forma independiente pueden capturar mejor la relación entre los datos y reducir de esta forma la varianza en comparación a un solo árbol de decisión.

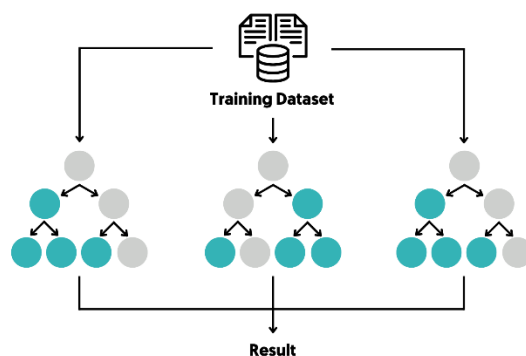


Fig. 20 Visualización de un Random Forest Tree

Este método de utilización se considera una gran mejora sobre los árboles de decisión ya que tienen a tener problemas de “overfitting” fácilmente. Al disponer de varios con distintos parámetros y varios árboles se puede promediar las predicciones y lograr un consenso de predicción más acertada.

El objetivo nuevamente es verificar el desempeño de un modelo más optimizado y robusto para solucionar el problema de análisis y determinar cómo optimizar el primer modelo.

B. Modelo

El modelo de Random Forest Tree fue implementado con la librería de “Sklearn”, la cual proporciona una gran cantidad de herramientas robustas para la construcción y evaluación de modelos ML. Los conjuntos de datos se dividieron de la siguiente forma:

- “Train”: Utiliza el 70% de los datos
- “Validation”: Utiliza el 10% de los datos
- “Train”: Utiliza el 20% de los datos

El modelo que se utilizó fue RandomForestClassifier, así mismo los hiperparámetros se armaron en 5 configuraciones distintas, con el objetivo de encontrar la más optima.

Numero de árboles	Máxima profundidad
50	3
100	5
200	8
200	12
300	None

C. Resultados

Los hiperparámetros más óptimos fueron 200 árboles y una profundidad máxima de 12. A continuación se puede observar la matriz de confusión del mejor resultado.

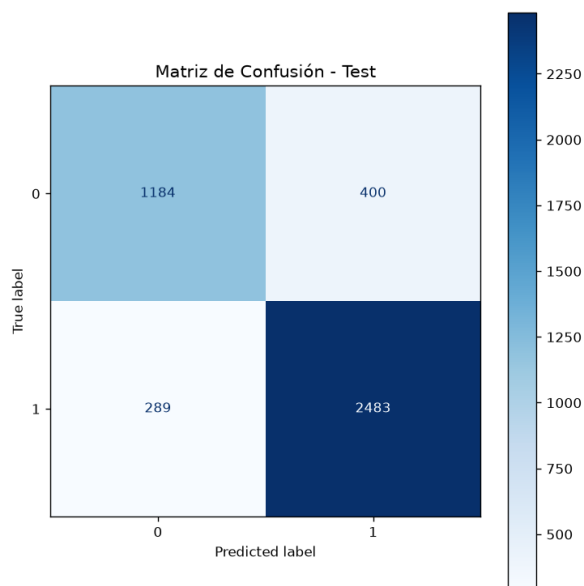


Fig. 21 Matriz de confusión de Random Forest Tree

Así mismo podemos ver la comparativa de las distintas configuraciones.

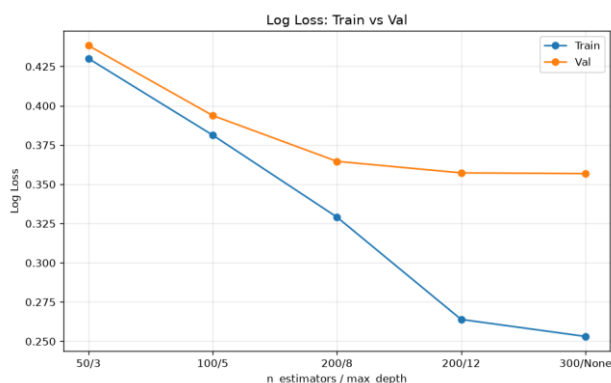


Fig. 22 Comparación de las configuraciones según Loss

Finalmente la precisión en test fue de 84.18% bastante parecido a los resultados del primer modelo

D. Interpretación

Al tener un resultado similar en la matriz de confusión y en la precisión del modelo al hacer la predicción teniendo en cuenta que ambos modelo tuvieron el conjunto de datos en la misma proporción así como el mismo set de datos podemos determinar que la optimización se debe realizar desde los datos.

Así mismo es importante mencionar que a medida que se incrementaban los hiperparámetros y el modelo se volvía más complicado se evidenciaba una situación de "overfitting" muy evidente, reforzando que usar un modelo más complejo no mejoraría la precisión.

Por lo que si ambos modelos lograron resultados similares probablemente los datos y las variables seleccionadas no cuentan la suficiente significancia para lograr incrementar la precisión. Es importante considerar como se puede optimizar el set de datos para lograr mejorar esa significancia y acercarnos un poco más a reducir el error dado por la función de costo.

VII. MEJORA AL PRIMER MODELO

Ya que todo apunta a que se necesita mejorar la significancia de las variables es importante decidir como se hará la mejora para poder correr la segunda versión del modelo.

Una opción de mejorar el set de datos con el que contamos y hemos estado trabajando es hacer un "One-Hot Encoding" la cuál es una técnica que permite convertir una columna categórica en varias columnas binarias, una por cada categoría posible, marcando 1 en la categoría que aplica a cada fila y 0 al resto.

Esto nos va a permitir que cada opción actúe como una variable independiente en la ecuación y así asignarle un peso propio a cada categoría. La idea es que si permitimos a cada categoría tener su propio peso eso ayudará a evitar que el modelo asuma una relación numérica falsa entre categorías.

Este "One-Hot Encoding" se aplicará a cada una de las columnas con categorías, estas serán:

- "Academic Pressure"
- "Study Satisfaction"
- "Financial Stress"
- "Sleep Duration"
- "Dietary Habits"

Nuevamente utilizando "pandas" como en la etapa de análisis exploratorio de datos, usaremos la función llamada ".get_dummies()" que permite hacer el "One-Hot Encoding" de forma rápida.

Esto generará un nuevo set de datos que cuenta con la cantidad de 30 columnas y 1 columna tipo "target", una mayor cantidad a las 11 columnas originales.

VIII. SEGUNDA VERSIÓN DEL MODELO

Con el nuevo set de datos se espera conseguir una mejora considerable para reducir el error que se encontraba en ambos modelos así como mejorar la precisión sin volver el modelo más complejo.

A. Parámetros

Pesos (w): Son inicializados en 0 y actualizados durante el entrenamiento dentro de la función de optimización.

Sesgo (b): Inicializado en 0 y actualizado durante el entrenamiento dentro de la función de optimización.

B. Hiperparámetros

Learning rate (lr): Taza de aprendizaje, controla que tan grandes son los pasos de la gradiente descendiente. Esta inicializado en 0.1.

Epochs (iteraciones): Número de iteraciones para ajustar los pesos del modelo. Una sola época equivale a una vuelta completa de los datos en la función de optimización.

IX. RESULTADOS DE LA SEGUNDA VERSIÓN DEL MODELO

A. Resultados

La segunda versión del modelo fue entrenado utilizando las mismas configuraciones que el primer modelo, 1000 iteraciones y 500 iteraciones con un “learning rate” de 0.1

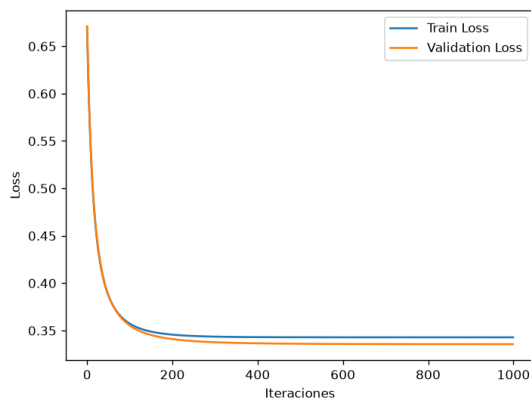


Fig. 23 Gráfica de costos train contra validation en 1000 iteraciones

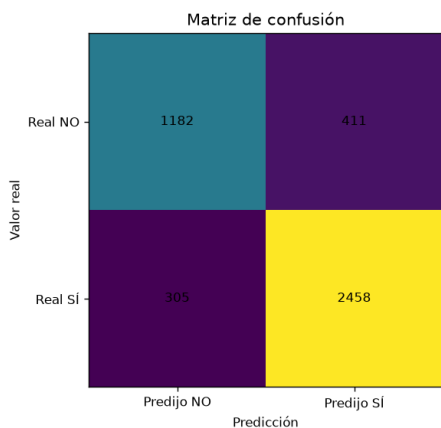


Fig. 24 Matriz de confusión de la segunda versión del modelo en 1000 iteraciones

La precisión de la segunda versión del modelo en 1000 iteraciones usando los datos de “test”: 83.56290174471994.

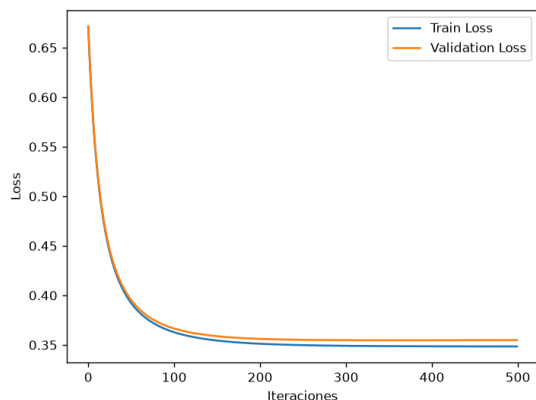


Fig. 25 Gráfica de costos train contra validation en 500 iteraciones

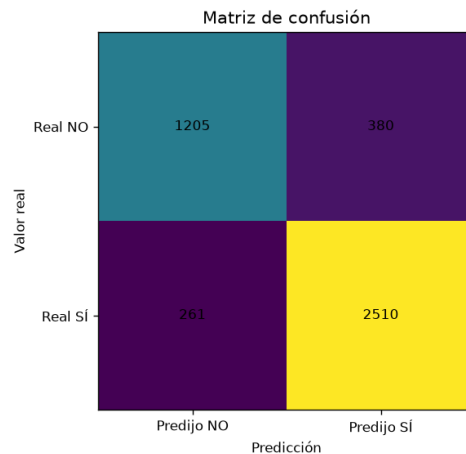


Fig. 26 Matriz de confusión de la segunda versión del modelo en 500 iteraciones

La precisión de la segunda versión del modelo en 500 iteraciones usando los datos de “test”: 85.28466483011937.

B. Interpretación

Como se puede observar en ambas configuraciones el cambio fue cercano a nulo, por lo que la segunda versión rinde lo mismo que la primera y por lo tanto no hay una mejora significativa al realizar “One-Hot Encoding” como se había planteado inicialmente, esto deja en claro que se necesita una aproximación distinta para lograr una mejora significativa.

C. Diagnóstico

Como ya se comprobó el ajustar las variables con las que se cuenta no hacen un cambio significativo se necesita hacer un ajuste distinto en el modelo.

Actualmente ambas versiones tienen un problema de “underfit” pues sus valores en la etapa de entrenamiento y validación respecto al costo son muy similares, pero ninguno de los dos logra bajar más del 0.34 de costo.

Sin embargo, al compararlo con los resultados de un modelo más robusto y conseguir resultados similares es probable que los datos no tengan la relación o significancia suficientes para poder predecir de forma correcta nuestra variable objetivo.

X. CONCLUSIÓN

A. Pensamientos finales

Como conclusión de esta investigación se puede conseguir una predicción con una precisión considerable utilizando un modelo de regresión, así como un “Random Forest Tree” sin embargo estos modelos no consiguen romper la barrera para lograr una predicción de 90% o más.

En esta investigación igualmente se creó y probó una hipótesis de mejora que no logró el resultado esperado al realizar un “One-hot Encoding” para mejorar la relación entre las variables y la variable objetivo.

B. Hacia donde avanzar

Dos opciones que pueden generar el resultado deseado son utilizar una red neuronal sencilla que pueda analizar el problema de una forma distinta o el ajuste de los hiperparámetros.

Durante esta investigación hubo un hiperparámetro que nunca cambio a lo largo de las versiones que fue el ritmo de aprendizaje o “*Learning rate*” que se sostuvo durante toda la investigación, es posible que reducir el ritmo de aprendizaje de 0.1 a 0.01 o 0.001 y aumentar el número de iteraciones pueda generar un efecto en el que se aproxime más al en la función de costo al contrario como ahora que se teoriza que podría estar oscilando, para esto se necesitan más pruebas.

C. Cierre

Los temas de salud mental no son algo que se deba tomar a la ligera es importante tratar estos temas con la seriedad necesaria. Es posible que mejoras a este modelo de aprendizaje pueda salvar vidas de cierta forma, al detectar depresión en base a patrones en el ritmo de vida de los estudiantes.

Sería interesante conocer sobre aproximaciones a este problema para lograr tener una precisión deseada sobre el objetivo. Solo la prueba y el error lograrán este resultado.

XI. REFERENCIAS

[1] ¿Qué es la regresión logística? - Explicación del modelo de regresión logística - AWS. (s. f.). Amazon Web Services, Inc. <https://aws.amazon.com/es/what-is/logistic-regression/>

[2] Sancho Caparrini, F. (s. f.). Entrenamiento de Redes Neuronales: mejorando el Gradiente Descendiente. Blog de Fernando Sancho. https://www.cs.us.es/~fsancho/Blog/posts/Mejorando_Gradiente_Descendiente.md

[3] ¿Qué es el descenso de gradiente? | IBM. (s. f.). IBM. <https://www.ibm.com/mx-es/think/topics/gradient-descent>

[4] ¿Qué es el machine learning? - Explicación del aprendizaje automático - AWS. (s. f.). Amazon Web Services, Inc. <https://aws.amazon.com/es/what-is/machine-learning/>

[5] Diferencia entre machine learning supervisado y no supervisado - AWS. (s. f.). Amazon Web Services, Inc. <https://aws.amazon.com/es/compare/the-difference-between-machine-learning-supervised-and-unsupervised/>

[6] Pykes, K. (2025, 16 enero). Función de pérdida de entropía cruzada en el aprendizaje automático: Mejorar la precisión del modelo. DataCamp. <https://www.datacamp.com/es/tutorial/the-cross-entropy-loss-function-in-machine-learning>

[7] Calvo, D. (2018, 10 diciembre). Función de coste - Redes neuronales. Diego Calvo. <https://www.diegocalvo.es/funcion-de-coste-redes-neuronales/>

[8] ¿Qué es Random Forest? | IBM. (s. f.). IBM. <https://www.ibm.com/mx-es/think/topics/random-forest>

[9] One Hot Encoding in Machine Learning. (s. f.). GeeksforGeeks. <https://www.geeksforgeeks.org/machine-learning/ml-one-hot-encoding>

[10] Student Depression Dataset. (s. f.). Kaggle. <https://www.kaggle.com/datasets/hopesb/student-depression-dataset/data>